

A Better Implementation: Numerical Stability

There is a more advanced and numerically stable way to implement neural networks for classification. This recommended method helps prevent a common issue called **numerical round-off error**.

The Problem: Numerical Round-off Error

Computers have a limited amount of precision to store numbers (known as floating-point numbers). Certain calculations can lead to small, accumulating errors.

- **Simple Analogy:**
 - **Direct (Accurate) Way:** $x = 2 / 10000$
 - **Indirect (Less Accurate) Way:** $x = (1 + 1/10000) - (1 - 1/10000)$
- Mathematically, these are identical. But in a computer, the "indirect" method can introduce tiny round-off errors.
- **Applying this to Neural Networks:**
 1. The "indirect" method is computing the activation $a = \text{sigmoid}(z)$ *first* and *then* feeding a into the loss function.
 2. The "direct" method involves giving the raw z value to the loss function and letting it perform a combined, more stable calculation.

The `from_logits` Solution

The recommended implementation involves setting `from_logits=True` in the TensorFlow loss function.

- **What are "logits"? Logits** are the raw, linear outputs of the final layer, z , **before** the activation function (like sigmoid or softmax) is applied.
 - **How it works:** By setting `from_logits=True`, you tell TensorFlow: "I am giving you the z values, not the a values. Please combine the activation function and the loss calculation into one mathematically stable step."
 - **Why is this better for Softmax?** The softmax function uses e^z . If z is a large number, e^z can become astronomically large (an "overflow"). If z is a large negative number, e^z can become vanishingly small (an "underflow"). The `from_logits=True` method uses mathematical tricks to compute the loss *without* computing these huge or tiny intermediate numbers, making it much more accurate.
-

Recommended TensorFlow Implementation

Here is a comparison of the standard (less stable) and recommended (more stable) methods.

Problem	Standard (Less Stable)	Recommended (More Stable)
Binary Classification	<pre>Dense(units=1, activation='sigmoid') model.compile(loss='binary_crossentropy')</pre>	<pre>Dense(units=1, activation='linear') model.compile(loss=BinaryCrossentropy(from_logits=True))</pre>
Multiclass Classification	<pre>Dense(units=10, activation='softmax') model.compile(loss='sparse_categorical_crossentropy')</pre>	<pre>Dense(units=10, activation='linear') model.compile(loss=SparseCategoricalCrossentropy(from_logits=True))</pre>

Important Consequence: Making Predictions

There is one critical side effect of using the recommended `from_logits` method:

- Your network's output is **no longer a probability**. The output layer now has a `linear` activation, so it outputs the raw **logits** (z).
- If you need to get a probability (e.g., to show to a user), you must **manually apply the activation function** to the model's output *after* making a prediction.
 - For logistic, you would pass the output z through a sigmoid function.
 - For softmax, you would pass the output vector $\vec{z}$ through a softmax function.